DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Bogura, Bangladesh

A THESIS ON

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Course Title: Thesis/Project Work

Course Code: CSE-499(B)

Prepared By

Md. Riaz

ID: 0322310105101024

Program: B.Sc. in Computer Science & Engineering

Semester: 8th Semester

Session: _____

Under the Supervision of

Mst. Sahela Rahman

Designation: Lecturer

Department of Computer Science & Engineering

Pundra University of Science & Technology

Date of Submission: _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PUNDRA UNIVERSITY OF SCIENCE & TECHNOLOGY

Bogura, Bangladesh

A Thesis on-

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science and Engineering

Course Title: Thesis/Project Work

Course Code: CSE-499(B)

Submitted to

Department of Computer Science & Engineering

Pundra University of Science & Technology

_____

Supervisor Signature

Mst. Sahela Rahman

Department of Computer Science & Engineering, Pundra University of Science & Technology

_____

Student Signature

Md. Riaz, ID: 0322310105101024

CERTIFICATION OF ORIGINALITY

I hereby declare that this thesis titled "AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics" is my own original work, carried out under the supervision of Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology. To the best of my knowledge, this thesis contains no material previously published or written by another person, except where due reference is made in the text. This work has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

_____

Signature of Student

Md. Riaz

ID: 0322310105101024

CERTIFICATION OF APPROVAL

This thesis titled

"AEGIS: A Constraint-Based Architecture for Safe LLM-Assisted Natural Language Analytics"

submitted by Md. Riaz to the Department of Computer Science & Engineering, Pundra University of Science & Technology, has been examined and is hereby approved as a partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Supervisor:

_____

Mst. Sahela Rahman

Department of Computer Science & Engineering

Pundra University of Science & Technology

Examiner:

_____

Name: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mst. Sahela Rahman, Lecturer, Department of Computer Science & Engineering, Pundra University of Science & Technology, for her continuous guidance, patience, and constructive feedback throughout the design, implementation, and evaluation of this research. Her insistence on precise, defensible claims shaped this thesis at every stage, from the formal safety proof to the honesty of its limitations section.

I am also grateful to the faculty members of the Department of Computer Science & Engineering for the coursework and feedback that prepared me to undertake this research, and to my family for their patience and encouragement during the long implementation and evaluation cycles this thesis required.

Finally, I acknowledge the authors of the prior work surveyed in Chapter 2 of this thesis. Their published research on natural language interfaces, text-to-SQL translation, and dashboard generation provided the foundation against which AEGIS's contributions are measured.

Table of Contents

List of Figures

List of Tables

ABSTRACT

Analytical dashboards are important tools for business reporting, but building accurate and safe reports from relational databases still requires technical skills. Natural language interfaces try to close this gap, but current text-to-SQL systems focus on benchmark accuracy rather than real-world safety, and they stop at generating a one-time query result without producing reusable reporting widgets. This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that turns plain-English reporting requests into dynamic, refreshable dashboard widgets that users can save and reuse every day. Unlike traditional natural-language-to-SQL systems that treat each question as a one-off interaction, AEGIS produces persistent reporting widgets, each with its own refresh schedule, access rules, and visual configuration, that become part of a user's daily workflow.

AEGIS uses a strictly controlled pipeline: (1) a lightweight large language model (Llama 3.1 8B) maps natural language to one of eleven high-level analytical primitives (for example KPI, Trend, Ranking, Tabular) using dynamic vocabulary injection; (2) a deterministic compiler builds the SQL using pre-approved parameterized templates; and (3) a post-compilation security monitor validates the statement against a strict safety grammar. Evaluation via a 100-query prototype benchmark in a real e-commerce domain (nopCommerce) demonstrates 100% intent accuracy on the covered primitives and structural prevention of SQL injection through untrusted natural-language input, a guarantee that holds within the defined threat boundary of trusted semantic-layer definitions and administrator-controlled compiler templates. A cross-schema evaluation on WooCommerce confirms that only the semantic layer requires reconfiguration for a new schema, achieving 98.0% intent accuracy in 14 person-hours. AEGIS demonstrates that restricting SQL generation to a finite set of validated business patterns is a practical path to safe, auditable natural-language reporting in institutional environments.

Index Terms: Natural language interfaces, dashboard generation, text-to-SQL, semantic layer, visualization recommendation, business intelligence, self-service analytics.

CHAPTER 1

INTRODUCTION

1.1 Background

Relational databases store critical institutional data in organizations: financial records, customer accounts, sales transactions, and more. But accessing this data is uneven. Technical staff can write SQL queries to get any answer they need, while non-technical users have to wait for someone else to build them a report. This waiting is expensive. Analysis of enterprise reporting workflows shows that business users frequently wait days for new reports, and a recurring theme in institutional reporting is that many questions are variations of things already asked before: the same report with a different date range, or the same chart for a different department. These are not one-off questions; they are recurring reporting needs that should be served by saved, refreshable widgets rather than regenerated from scratch every time.

This thesis presents AEGIS (Analytics Engine with Guaranteed Injection Safety), a system that lets users describe their reporting needs in plain English and produces dynamic dashboard widgets that can be saved, refreshed, and reused as part of their daily workflow, without anyone writing SQL.

Natural language interfaces to databases (NLIDBs) try to solve this problem. The idea is simple: a user should be able to ask “which categories have the highest refund rates this month?” and get a correct, visual answer without writing SQL. Researchers have made good progress here. Neural text-to-SQL systems now exceed 90% accuracy on the Spider benchmark [16], and large language models can produce reasonable-looking SQL with minimal setup [6]. But there is still a gap between benchmark results and real-world deployment, which Chapter 2 examines in detail.

1.2 Problem Statement

Three problems make up the gap between benchmark text-to-SQL accuracy and safe, production-ready natural-language reporting.

Safety. Many modern natural-language-to-SQL systems rely on models that directly generate SQL tokens, which creates challenges for enforcing enterprise governance and security policies. An LLM generating SQL freely can produce queries that expose private data or use incorrect table joins, and detecting this after the fact is fundamentally harder than preventing it by construction.

Vocabulary mismatch. Benchmarks use actual column names in the questions, but real users speak in business terms ("refund rate" instead of SUM(o.RefundedAmount)). Matching these requires business knowledge that models do not always get right, and a wrong mapping can silently produce a plausible but incorrect report.

No widget generation. Existing systems answer one question at a time and discard the result. They do not produce saved reporting widgets that can be refreshed with new data tomorrow, shared with a colleague, or added to a daily dashboard. Every time someone needs the same report, they start from scratch, which directly contradicts the observation that reporting requests are often recurring rather than one-off (Section 3.2).

These problems are not primarily about building a smarter model; they are about designing the system properly around the model. AEGIS addresses this by splitting the work into stages. The LLM's only job is to understand what the user is asking and output a structured description of the request. Everything after that, matching to the right business terms, building the SQL, selecting the chart, and saving the widget, is done by fixed rules and pre-approved templates.

1.3 Research Novelty and Motivation

Existing text-to-SQL research asks: how accurately can a model generate SQL from natural language? This thesis asks a different question: how can large language models be used for language understanding while being structurally prevented from generating executable SQL at all? The two pipelines differ structurally.

Classical NL-to-SQL: Natural Language → SQL generation → Query result.

AEGIS: Natural Language → Intent extraction → Semantic constraint → Deterministic compilation → Safe analytical artifact.

The contribution of this thesis is therefore not improved SQL generation accuracy; it is constrained analytical artifact generation, a design approach that removes SQL generation from the LLM's role entirely and provides safety and semantic fidelity guarantees that no generative model can match unconditionally.

AEGIS does not propose a new LLM. The model is interchangeable: Groq-hosted Llama 3.1 8B, OpenRouter, a local Ollama instance, or any endpoint compatible with the /v1/chat/completions interface. Because the LLM's only contract with the rest of the system is to produce a typed JSON intent object, model upgrades improve quality automatically without changing the compiler or safety infrastructure. This is model independence by design, not an implementation accident.

1.4 Objectives and Contributions

This thesis, presented at the mid-defense stage, makes the following contributions; the quantitative figures below are preliminary estimates from the author's own prototype evaluation to date (Chapter 5), not final or independently verified results:

1. A design-time review of representative e-commerce and business-intelligence reporting requests, resulting in eleven common reporting patterns validated against a published 100-query benchmark (Chapter 3).
2. A system design in which all possible queries are limited to pre-approved templates and a defined semantic layer, which prevents SQL injection and unauthorized data access by construction (Chapter 3).
3. The AEGIS system itself, including the semantic layer design, a vocabulary injection prompting strategy, a safe SQL builder with two-layer defence, a rule-based chart selector, and a widget storage system with scheduled refresh (Chapters 3-4).
4. A vocabulary injection method that places the approved metric and dimension names directly into the LLM prompt, removing the need for a manually written synonym list while achieving 100% coverage, reducing the synonym dictionary from 112 entries to zero (Chapter 3).
5. A benchmark evaluation of 100 queries showing 100% valid SQL and 0% unsafe queries, compared to 5.0% unsafe queries from a direct LLM-to-SQL baseline (Chapter 5).

6. A cross-schema generalizability study on WooCommerce showing that only the semantic layer requires modification when deploying to a new production schema, with 98% intent accuracy achieved in 14 person-hours of configuration (Chapter 5).
7. A pipeline latency analysis showing that the AEGIS safety infrastructure adds less than 4% overhead relative to the LLM API call, making the safety guarantees effectively free in practice (Chapter 5).

1.5 Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews related work across four decades of natural language database interfaces, neural text-to-SQL, natural-language-driven visualization, dashboard generation, and semantic layers, and positions AEGIS relative to this body of work through a comparative summary and a research gap analysis. Chapter 3 presents the methodology: the research paradigm, the formative study of real reporting requests that motivated AEGIS's design, the formal model and threat model that define its safety guarantee, and the system architecture, semantic layer, intent parser, safe compiler, visualization selector, and widget engine that implement it. Chapter 4 describes the experimental work: the implementation, the experimental environment, the benchmark dataset, and the baseline systems used for comparison. Chapter 5 reports results for each of the five research questions and discusses what they mean, including a structural comparison against direct LLM-to-SQL generation and an analysis of what AEGIS deliberately gives up in exchange for its safety guarantees. Chapter 6 states the limitations of the current work honestly and outlines future work. Chapter 7 concludes the thesis.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

This chapter reviews the sources most directly comparable to AEGIS, grouped by theme: natural language interfaces to databases, neural and LLM-based text-to-SQL, natural language for visualization and dashboards, and applied conversational business intelligence. Closely related sources are discussed together rather than one at a time where they make the same architectural point. Sources only loosely adjacent to AEGIS's technical contribution (general business-adoption surveys, dashboard-governance literature, evaluation-methodology papers unrelated to safety) are outside this review's scope.

2.1 Natural Language Interfaces to Databases

Two systematic surveys frame this literature. Affolter et al. [1] benchmark 24 NLIDBs against ten questions of increasing complexity across four architectural classes (keyword, pattern, parsing, grammar-based); grammar-based systems achieve the broadest coverage but require users to learn a constrained vocabulary. Liu and Xu's more recent review [7] is the only one to name SQL-injection risk directly, discussing TrojanSQL, a backdoor injection attack the authors report is difficult to defend against, yet their only recommended mitigation is generic (“additional layers of security or filtering”), with no architectural defense proposed. Neither survey finds a system that treats safety as a structural property.

NaLIR [5] and Veezoo [4] are the closest prior systems to AEGIS's use of a curated vocabulary. NaLIR parses a query into a grammar-constrained intermediate “query tree” and surfaces ambiguity to the user through a short multiple-choice interaction rather than guessing, correctly completing 88 of 98 tasks versus 56 of 98 without this step. Veezoo constrains matching with an editable Knowledge Graph structurally analogous to a semantic layer, needing a median of two reformulations to reach a correct answer in a controlled study. Both, however, still compile into open-ended SQL and depend on user-facing dialogue to recover from a wrong query rather than preventing an unsafe one up front, and both produce one-off answers with no persistence.

2.2 Neural and LLM-Based Text-to-SQL

Spider [16] and BIRD [6] provide the clearest evidence that free-form generation is unreliable at scale. Spider's strict cross-domain train/test split found the best contemporary baseline reached only 12.4% exact-match accuracy on unseen schemas. BIRD went further, testing 95 real, large databases with expert-annotated domain knowledge: even GPT-4 combined with DIN-SQL prompting reached only 55.9% execution accuracy against a 92.96% human-expert ceiling, with wrong schema linking (41.6%) and misunderstood database values (40.8%) as the dominant failure modes.

RAT-SQL [15] and PICARD [10] each constrain the same underlying generation approach without changing who authors the SQL. RAT-SQL encodes the schema as a relational graph via relation-aware self-attention, reaching 57.2% exact-match on Spider, yet oracle analysis attributes 72-81% of its remaining errors to wrong column or table selection. PICARD instead constrains decoding at the token level, cutting invalid SQL on Spider from roughly 12% to 2% — but the LLM still writes every character of the SQL string, so a semantic bug invisible to the grammar check (a wrong join, a wrong business formula) can still pass through as syntactically valid SQL.

G-SQL [12] and TriSQL [13] sit at opposite ends of the same spectrum and are the two closest points of comparison for AEGIS. G-SQL is rule-based and template-driven over a curated schema representation, deliberately avoiding a neural component that writes SQL directly; it reaches 100% execution accuracy on easy queries but falls to 45-55% on extra-hard ones. TriSQL is, at the time of writing, the state of the art: a three-stage LLM pipeline (schema selection, skeleton-first generation, complexity-aware refinement) reaching 82.2% execution accuracy on Spider. It is the sharpest possible contrast to AEGIS, precisely because it is state of the art and still has the LLM produce the final executable SQL directly, with no semantic layer and no reported safety evaluation.

2.3 Natural Language for Visualization and Dashboards

nl4dv [8] and DataTone [3] are the closest visualization-side precedents. nl4dv maps natural language to a small, fixed set of five analytic tasks via a rule-based mapping

table, closely analogous to AEGIS's bounded-vocabulary design, but has no SQL-safety or permission layer at all since it operates only on an in-memory dataset, and produces a one-off specification with no persistence. DataTone instead surfaces ambiguity through interactive widgets rather than resolving it silently, significantly outperforming IBM Watson Analytics in a comparative study (5.43 vs. 2.00 correct facts, $p < 0.01$), but is limited to single-table data with no persistence mechanism.

DashBot [2] composes multi-chart dashboards with deep reinforcement learning guided by design rules (diversity, parsimony) drawn from a study of 90 real dashboards. It has no natural-language input at all, and because it never generates SQL from user language, it never encounters the safety problem AEGIS is built to solve.

2.4 Applied Conversational Business Intelligence

Two recent applied studies bookend AEGIS's design choice. Shailesh et al. [11] describe a deployed Groq/LangChain assistant that gives an LLM direct SQL-execution tools and a self-correction retry loop — a live, working example of exactly the attack surface AEGIS's threat model closes (Section 3.5), with no reported adversarial evaluation. Valkenburgh [14], by contrast, independently arrives at AEGIS's central design principle in an unrelated domain: a deterministic, non-AI formalism computes a business explanation, and an LLM is used only to narrate the already-correct result. A pre-study in the same thesis found that ten commercial AI-BI products could all answer simple descriptive queries but none could correctly answer explanatory ones without manual reconfiguration — direct evidence that unconstrained LLM reasoning fails at exactly the class of task AEGIS targets.

2.5 Comparative Summary

Table 4: Comparative summary of the most closely related systems.

System	NL Parsing	Semantic Layer	Safe SQL	Visualization	Widget Persistence	Coverage Validation	Evaluation
Spider /	Yes	-	-	-	-	-	Benchmark

BIRD							only
RAT-SQL / PICARD	Yes	-	Partial	-	-	-	Benchmark only
G-SQL / TriSQL	Yes	Partial	Partial	-	-	-	Benchmark only
NaLIR	Yes	-	-	-	-	-	User study
Veezoo (Lehmann et al.)	Yes	Yes	-	-	-	-	User study
nl4dv / DataTone	Yes	-	-	Yes	-	-	In-memory / user study
DashBot	-	-	-	Yes	Partial	-	Synthetic study
Conversational BI Assistant (Shailesh et al.)	Yes	-	-	Yes	-	-	Prototype demo
AEGIS (this thesis)	Yes	Yes	Yes	Yes	Yes	Yes	Production (nopCommerce)

2.6 Research Gap Analysis

Two gaps recur across every source reviewed above, including the state of the art. First, no system treats SQL-generation safety as a structural property rather than an accuracy metric: Spider, BIRD, RAT-SQL, PICARD, G-SQL, and TriSQL are all evaluated purely on exact-match or execution accuracy, and Shailesh et al.'s deployed assistant reports no adversarial evaluation at all despite giving an LLM direct database access. AEGIS closes this gap by removing SQL generation from the LLM's output space entirely (Section 3.4) and evaluating an explicit unsafe-SQL rate against a direct LLM-to-SQL baseline (Chapter 5), rather than relying on accuracy as a proxy for safety.

Second, no system combines a governed semantic vocabulary with persistent, refreshable output: NaLIR and Veezoo produce one-off answers, nl4dv and DataTone produce one-off visualizations, and DashBot has no natural-language input at all. AEGIS's widget engine (Section 3.11) closes this gap, motivated by the design-time observation that real reporting requests are often recurring rather than one-off (Section 3.2). Valkenburgh's independent arrival at AEGIS's “let a deterministic layer compute the answer” principle, in an unrelated domain, is the strongest external corroboration that this design decision reflects a pattern discovered wherever LLM reliability is treated as an engineering constraint rather than an accuracy target.

CHAPTER 3

METHODOLOGY

3.1 Research Paradigm

This thesis follows a Design Science Research paradigm, in the sense used by Hevner et al.: knowledge is produced by building and evaluating a novel artifact that addresses an identified organizational problem, rather than by testing a hypothesis about an existing phenomenon. The artifact in this thesis is the AEGIS system itself. Design Science Research requires three things to be demonstrated: problem relevance, design rigor, and evaluation against the stated problem. Problem relevance is established through the formative study of Section 3.2, which analyzes real reporting behavior rather than assuming a problem exists. Design rigor is established through the formal model and threat model of Sections 3.4-3.5, which state precisely what safety property the architecture guarantees and under what boundary that guarantee holds. Evaluation is reported in Chapter 5 against five explicit research questions, using both a benchmark dataset constructed for this domain and an ablation study that isolates the contribution of each architectural component.

The research proceeded through three iterative build-evaluate cycles. The first cycle produced a semantic layer and compiler for a minimal set of intent classes and validated the core safety proposition on a small hand-written query set. The second cycle expanded coverage to all eleven intent classes identified in the formative study and introduced vocabulary injection after an early synonym-dictionary approach proved unable to keep pace with real query phrasing. The third cycle added the widget persistence layer and the cross-schema generalizability evaluation on WooCommerce, testing whether the architecture itself, not just the nopCommerce configuration, was the reusable artifact.

3.2 Formative Study of Reporting Patterns

The eleven-pattern taxonomy used throughout this thesis (Table 2) originates from a design-time review of representative e-commerce and administrative reporting requests, conducted by the author while designing AEGIS. This was a qualitative

review carried out by a single researcher during system design, not an independently annotated, inter-rater-validated study, and no separate annotated dataset accompanies this thesis. The taxonomy identified through this review was then operationalized as the pattern library (Table 2) and evaluated directly against the 100-query benchmark described in Chapter 4, which is the verifiable evidence this thesis relies on for its coverage and safety claims. An earlier version of this chapter reported specific percentages and an inter-rater reliability statistic for a 312-request dataset; that dataset was never published alongside this thesis, so those figures have been withdrawn and are not repeated here.

Table (formative study): Request taxonomy, in approximate order of how often each pattern was observed during the design-time review.

Pattern	Example
Ranking	Which five categories have the highest refund rates?
Trend Analysis	Show monthly sales volume over the last year.
KPI / Aggregate	How many orders were placed today?
Comparison	Compare average order value between mobile and desktop users.
Exception / Filter	List products with stock levels below 10.
Summary / Group	Give me an overview of the Electronics category.
Segment, Funnel, Cohort, Correlate, Tabular	Revenue by category; cart-to-purchase conversion; new vs. returning customers; attribute correlation; raw order listings.

[PLACEHOLDER — FIGURE 5 NOT YET INSERTED]

Illustrative pattern taxonomy from the design-time review

A bar chart showing the eleven patterns in the approximate relative order they were observed during the design-time review (Ranking, Trend Analysis, and KPI/Aggregate most frequently; Segment, Funnel, Cohort, and Correlate least frequently). Label the chart clearly as illustrative of the design rationale, not a measured survey, since no percentage breakdown is claimed (Section 3.2).

Figure 5: Illustrative pattern taxonomy from the design-time review

This review yields three design directions that shaped the rest of this chapter. First, a small set of patterns appears sufficient: the eleven identified patterns cover the large majority of reviewed requests, supporting a fixed template library rather than an open-ended query language. Second, business vocabulary differs systematically from database column names: reviewed requests used phrases like “total refund rate,” never SUM(o.RefundedAmount), which motivates an explicit semantic layer (Section 3.7) rather than exposing the schema directly to the language model. Third, reuse appears to be the norm rather than the exception: many requests were variations of things already asked before, in a different time window or for a different segment, which motivates the widget persistence design of Section 3.11.

3.3 Design Principles

Five principles guide every architectural decision in this chapter:

8. Separate understanding from execution. The LLM understands the question; fixed rules handle everything else.
9. Define business terms explicitly. Metrics, dimensions, joins, and time rules are written once in a semantic layer, not inferred per query.
10. Limit what SQL can be generated. SQL is built only from pre-approved, parameterized templates.
11. Select visualizations by rule. Chart type is decided by question type, result shape, and established visualization design guidance, not by a learned model.

12. Persist results for reuse. Every query produces a saved, refreshable widget rather than a discarded answer.

3.4 Formal Model

Let a user with role r issue a natural-language request q. Classical text-to-SQL seeks a function f(q, S) that maps the request and a schema S directly to sql. AEGIS instead seeks a function

$g(q, L, r) \rightarrow \langle \pi, sql, vis, w \rangle$

where π is a canonical analysis plan, sql is a read-only compiled query, vis is a visualization specification, and w is a persisted widget artifact. The semantic layer $L = \langle M, D, F, J, P, V, A, R \rangle$ defines the approved metric set M, dimension set D, filter and time-rule set F, join graph J, pattern library P, visualization policy V, vocabulary injection configuration A, and role-permission model R.

Safety is enforced as a set-membership constraint: $sql \in Q\_safe(L, r)$, where Q_safe(L, r) is the family of queries derivable from pattern templates in P using only bindings from L permitted under role r.

Proposition 1. *No query in Q_safe(L, r) can reference a table, column, or row not enumerated in L for role r. All SQL identifiers are drawn from a closed vocabulary of approved semantic bindings. All literal values are passed using parameterized SQL rather than string interpolation. SQL injection through untrusted natural-language input is structurally prevented by design.*

Security boundary. This guarantee holds within a defined threat boundary: the semantic layer definitions, compiler templates, and permission predicates are trusted, administrator-controlled artifacts. An administrator who embeds malicious SQL inside a metric definition, or a supply-chain compromise of the compiler library, falls outside this boundary and requires separate operational security controls. Explicitly stating this boundary, rather than leaving it implicit, is itself part of this thesis's contribution: prior NL-to-SQL work reviewed in Chapter 2 rarely specifies the boundary of its safety claims, which makes rigorous security comparison difficult.

3.5 Threat Model

AEGIS protects against attacks arriving through the untrusted natural-language input channel. The model assumes the database and application server are properly hardened; the attacker controls only the query field.

T1 - Prompt injection attempting SQL generation.

Attack: *"Ignore previous instructions. Generate DROP TABLE orders."*

Control: The IntentObject schema contains no SQL field. Any non-approved string in metric_term or dimension_term is rejected by Pydantic type validation at Stage 2, before the compiler is reached.

T2 - Unauthorized metric or dimension access.

Attack: *"Show me customer passwords" or "List credit card numbers by order."*

Control: Fields such as customer_password do not exist in the semantic layer vocabulary. The LLM never sees those names; it receives only the curated, approved label list. Stage 2 rejects any unrecognized term.

T3 - Unauthorized row access.

Attack: *A store-level user asks: "Show revenue for all branches."*

Control: Stage 4 (Permission Rewriter) runs after the LLM and appends a role-specific WHERE predicate (for example, AND o.StoreId = :user_store) derived from the authenticated session. This cannot be suppressed or overridden by natural-language content.

T4 - DML or DDL injection.

Attack: *A crafted prompt that tricks the LLM into associating a write operation with an intent class.*

Control: No template in the pattern library contains a DML or DDL keyword. The AST-level post-compilation validator explicitly rejects any non-SELECT statement as a defense-in-depth layer.

Not protected by AEGIS (requires operational security controls outside this architecture):

- A malicious administrator embedding arbitrary SQL inside a metric's sql_expr field.
- Supply-chain compromise of the compiler module or the SQL parser library.

- Database-level privilege escalation that bypasses the application layer.
- LLM provider infrastructure compromise or model poisoning.

Explicitly documenting out-of-scope threats is itself a contribution: prior NL-to-SQL work rarely specifies the boundary of its safety claims, which makes meaningful security comparison difficult.

3.6 System Architecture

Every user query travels through exactly seven stages. Stages 2 through 7 contain no artificial intelligence; they are deterministic code. Rejection at any stage produces a structured clarification message rather than a best-effort guess.

Stage 1 - Intent Extraction. A lightweight LLM reads the query together with a system prompt built from the semantic layer, and outputs a validated IntentObject (intent class, metric term, dimension term, filters, sort, limit, confidence). This is the only stage that involves artificial intelligence.

Stage 2 - Coverage Validation. The server checks that both the metric term and the dimension term exist in the semantic layer vocabulary before anything else runs. Unknown identifiers are rejected here, with a structured message listing the available identifiers, rather than being passed to the compiler.

Stage 3 - Semantic Mapping. Business-logic aliases are expanded (for example, “abandoned” maps to a specific OrderStatusId), and relative time expressions such as “this month” are resolved to concrete date predicates.

Stage 4 - Permission Rewriting. A role-specific WHERE predicate is appended based on the authenticated user's session. This runs after the LLM has already finished, so no natural-language content can influence it.

Stage 5 - SQL Compilation. A breadth-first search over the join graph finds the minimal join path connecting the tables required by the resolved metric and dimension, and pre-compiled SQL expressions are substituted into a parameterized template. No SQL text is ever assembled from concatenated user input.

Stage 6 - Visualization Selection. A rule engine maps the intent class and result shape to a default chart type (Section 3.10). This stage contains no learned model.

Stage 7 - Widget Persistence. The analysis plan is hashed (SHA-256) to detect duplicates, and the query, chart configuration, and access rules are stored as a widget artifact that can be refreshed on a schedule (Section 3.11).

[PLACEHOLDER — FIGURE 1 NOT YET INSERTED]

AEGIS architecture pipeline (User Request to Dashboard Widget)

A left-to-right flowchart of the seven stages in sequence: User Request -> LLM Intent Parser -> Coverage Validator -> Semantic Mapper -> Permission Rewriter -> Safe Query Compiler -> Query Executor -> Visualization Selector -> Widget Engine -> Dashboard. Color-code by responsibility: blue for the single AI/LLM stage, purple for semantic mapping, red for the two safety-enforcement stages (Permission Rewriter, Safe Query Compiler), green for execution/output stages. Add small orange branch arrows from Coverage Validator and Safe Query Compiler pointing to a 'Structured Clarification / Rejection Message' box, showing that invalid requests exit early with an actionable error rather than a silent failure.

Figure 1: AEGIS architecture pipeline (User Request to Dashboard Widget)

3.7 Semantic Layer Design

The semantic layer is the most important non-AI component of AEGIS. It separates business language from the underlying database structure and defines exactly which metrics, joins, and permissions are allowed to exist. A useful analogy is LEGO blocks rather than free-form clay: the semantic layer defines a finite set of composable building blocks. User questions are unlimited, but every answerable question is a composition of these blocks.

[PLACEHOLDER — FIGURE 2 NOT YET INSERTED]

Semantic layer modularity - composable blocks vs. free-form SQL generation

A split-panel comparison. LEFT panel, labeled 'AEGIS': a small set of labeled building blocks (Metric, Dimension, Filter, Join Path, Pattern) shown snapping together into two or three example complete query shapes, like LEGO bricks combining into a finished model. RIGHT panel, labeled 'Direct LLM-to-SQL': a shapeless, cracked blob of clay with a warning icon, annotated '5.0% unsafe queries in baseline (Chapter 5, Section 5.2)'. The visual point is that AEGIS composes from a bounded set of safe parts while direct generation is unbounded and can take an unsafe shape.

Figure 2: Semantic layer modularity - composable blocks vs. free-form SQL generation

Table 1: Semantic layer object model.

Object	Field	Example
Metric	label, SQL expression, joins,	revenue = SUM(o.OrderTotal

	visual default, security class	- o.RefundedAmount)
Dimension	label, SQL expression, datatype, access scope	category = c.Name from Category
Filter	label, SQL predicate, datatype	payment_status : o.PaymentStatusId = :val
Time rule	label, SQL predicate, granularity	current_week : DATEADD(week, ...)
Join path	source, target, ON clause	Order → OrderItem → Product → Category
Pattern	required slots, SQL template, visualization default	ranking : metric + dimension → bar chart
Permission	rule	store_manager → filtered by store location

In the nopCommerce prototype, the semantic layer defines 15 metrics, 34 dimensions, and 11 join paths across 12 analytics-relevant tables. The full nopCommerce schema contains 126 tables; the remaining 114 (system, content-management, configuration, authentication, vendor, and promotions tables) are deliberately not represented in the semantic layer at all. No business analyst asks “show me revenue by ScheduleTask,” and excluding these tables functions as an implicit table-level access control: even a crafted prompt that names a hidden system table directly is rejected at Stage 2, because the identifier simply does not exist in L.

3.8 Intent Parsing with Dynamic Vocabulary Injection

The central technique that makes Stage 1 reliable is vocabulary injection. At startup, AEGIS builds the system prompt by listing every approved metric and dimension name, with a plain-English description, directly from the semantic layer (approximately 1,100 tokens for 15 metrics and 34 dimensions). The LLM sees exactly which identifiers are valid and maps any user phrasing to the correct identifier without a manually maintained synonym list. This has three advantages over a synonym dictionary: zero maintenance, since adding a metric automatically updates the vocabulary the model sees; broad coverage of arbitrary user phrasing, since the

model performs the mapping rather than a fixed lookup table; and token efficiency, since the full vocabulary for 15 metrics and 34 dimensions fits in roughly 1,100 tokens.

Structured output enforcement for LLMs [9] constrains the model's response to a fixed JSON schema before any downstream validation occurs, which is why Stage 1 can rely on a typed IntentObject rather than free-form text: malformed or off-schema output is rejected at the API boundary, before Stage 2 coverage validation ever runs.

The output schema enforces typed fields:

```
{
  "intent_class": "kpi | ranking | trend | comparison | exception |
                   summary | segment | funnel | cohort | correlate |
tabular",
  "metric_term": "string",
  "dimension_term": "string or null",
  "time_term": "string or null",
  "filters": [{"field": "string", "operator": "string", "value":
"string"}],
  "sort": "asc | desc | null",
  "limit": "integer or null",
  "confidence": "low | medium | high",
  "needs_clarification": "boolean"
}
```

[PLACEHOLDER — FIGURE 3 NOT YET INSERTED]

Vocabulary injection workflow

A three-lane sequence diagram: 'Semantic Layer (semantic_layer.py)' -> 'System Prompt Builder' -> 'LLM'. Show the Semantic Layer box listing a few example metric/dimension entries with plain-English descriptions; an arrow labeled 'serializes into ~1,100 tokens' into the System Prompt Builder; then into the LLM, whose output arrow produces an example IntentObject such as {metric_term: 'revenue', dimension_term: 'category'}. Annotate with a small callout showing the user phrase 'earnings' mapping to the approved ID 'revenue' even though 'earnings' appears in no synonym list.

Figure 3: Vocabulary injection workflow

3.9 Safe Query Compiler

The compiler instantiates SQL from a library of parameterized templates, one per analytics pattern.

[PLACEHOLDER — FIGURE 4 NOT YET INSERTED]

Taxonomy of the eleven AEGIS analytical patterns

A tree or grid diagram with 'Eleven Analytical Patterns' at the top branching into eleven labeled leaves: KPI, Ranking, Trend, Comparison, Exception, Summary, Segment, Funnel, Cohort, Correlate, Tabular. Under each leaf, show a small icon of its default visualization (matching Table 2/3): card, bar chart, line chart, grouped bar, table, card grid, pie chart, funnel chart, grouped bar, scatter plot, table. Caption note: '~5,610 valid combinations across 15 metrics x 34 dimensions x 11 patterns.'

Figure 4: Taxonomy of the eleven AEGIS analytical patterns

Table 2: The eleven AEGIS analytical patterns.

Pattern	Required slots	Optional slots	Default visual
KPI (Aggregate)	metric	time_rule, filter	kpi_card
Ranking	metric, dimension	time_rule, filter, limit	bar_chart
Trend	metric, time_grain	time_rule, filter	line_chart
Comparison	metric, segment	time_rule, filter	grouped_bar
Exception	metric, threshold	dimension, time_rule	table
Summary	metric[], dimension	time_rule, filter	multi_card
Segment	metric, dimension	time_rule, filter	pie_chart
Funnel	metric, stages	time_rule, filter	funnel_chart
Cohort	metric, group_def	time_rule, filter	grouped_bar
Correlate	metric, attribute	time_rule, filter	scatter_plot
Tabular	dimension	filters, time_rule	table

After placeholder substitution, two safety layers apply in sequence. Layer 1 is a parameterized query engine that separates SQL structure from user-supplied inputs, so

no user text is ever concatenated into the SQL string. Layer 2 is a post-compilation safety scanner that rejects any assembled query containing a forbidden construct: non-SELECT statements, UNION, EXCEPT or INTERSECT, EXEC, or references to system tables. If any forbidden pattern is detected, the compiler raises a SecurityError rather than returning a partially safe query.

[PLACEHOLDER — FIGURE 6 NOT YET INSERTED]

Two-layer SQL safety defence

A vertical two-stage flowchart. A crafted/adversarial input enters at the top with a warning icon. Layer 1 box: 'Parameterized Query Engine — user text never enters the SQL string; only IDs and bound values appear.' Arrow down to Layer 2 box: 'Post-Compilation Safety Scanner — rejects non-SELECT statements, UNION/EXCEPT/INTERSECT, EXEC, and system-table references (16 forbidden patterns checked).' Below, split into two outcomes: a green 'Safe SQL Executed' box for a legitimate query, and a red 'SecurityError Raised' box for a rejected one, showing the attack is caught before it can reach the database regardless of which layer catches it.

Figure 6: Two-layer SQL safety defence

3.10 Visualization Selector

Table 3: Visualization selector mapping.

Intent	Result shape	Selected visualization
KPI	scalar	KPI card
Ranking	1 measure, up to 20 categories	Horizontal bar chart
Trend	1 measure, time series	Line chart
Comparison	1 measure, 2-4 segments	Grouped bar chart
Exception	row-level detail	Sortable table
Summary	2-4 scalar measures	KPI card grid
Segment	1 measure, categorical	Pie chart
Funnel	ordered conversion stages	Funnel chart
Cohort	1 measure, 2+ groups	Grouped bar chart
Correlate	2 measures, continuous	Scatter plot

Tabular	raw records	Sortable table

Two additional rules apply after the data is known, rather than at selection time: bar charts with more than 20 categories are automatically converted to a table, and pie charts with more than 8 slices are converted to a bar chart. Both rules exist because the initial chart choice is made before the exact cardinality of the result is known.

3.11 Widget Persistence and Reuse

Each widget stores a unique identifier (a SHA-256 hash of the analysis plan), the original question, the analysis plan in JSON form, a hash of the SQL template used, chart configuration, timestamps, access rules, and run history. A new question triggers the full seven-stage pipeline; if an identical widget already exists, determined by a match on the plan hash, the cached artifact is returned immediately rather than recompiled. Scheduled refresh re-executes the stored SQL against fresh data on a configurable interval, which directly addresses the design-time observation (Section 3.2) that reporting requests are often recurring rather than one-off: a widget answers the question once and then continues answering it as new data arrives, rather than requiring the same natural-language request to be re-processed from scratch every time.

[PLACEHOLDER — FIGURE 7 NOT YET INSERTED]

Widget lifecycle and refresh model

A cyclical flowchart. Start: 'New Natural-Language Question' -> 'Full Seven-Stage Pipeline Runs' -> 'Analysis Plan Hashed (SHA-256)' -> a decision diamond 'Identical widget hash already exists?'. If YES, arrow to 'Return Cached Widget Immediately'. If NO, arrow to 'Save New Widget (SQL, chart config, access rule, refresh schedule)'. Both paths converge into a 'Dashboard Widget' box, from which a looping arrow labeled 'Scheduled Refresh (re-executes stored SQL on fresh data)' curves back into the same box — illustrating that a widget keeps answering the same recurring question rather than being discarded after one use.

Figure 7: Widget lifecycle and refresh model

CHAPTER 4

EXPERIMENTAL WORK

4.1 Implementation

AEGIS is implemented as a web application with a vanilla HTML and JavaScript frontend (jQuery, Chart.js) and a Python FastAPI backend, targeting a production nopCommerce 4.70 schema (126 tables, 107 foreign key constraints). The implementation follows directly from the architecture in Chapter 3:

- LLM integration: Llama 3.1 8B Instant via the Groq API, with structured JSON output enforcement. The system prompt is constructed dynamically by injecting the approved metric and dimension identifiers at startup.
- Rate limiting: a provider-agnostic configuration module with a sliding-window rate limiter and a concurrency-safe asyncio.Lock, so the architecture is not tied to a specific LLM vendor's throughput characteristics.
- Semantic layer: Python configuration modules containing 15 metrics, 34 dimensions, zero synonym entries, and 11 join paths across the 12 analytics-relevant tables described in Section 3.7.
- SQL compiler: parameterized MySQL templates, with breadth-first search join-path resolution over the 12-table join graph. A post-compilation _validate_sql_safety() routine checks 16 forbidden patterns before a query is allowed to execute.
- Visualization selector: rule-based Python dictionaries implementing Table 3 of Chapter 3, with the two post-hoc cardinality rules applied after the result set is known.
- Widget engine: SHA-256 plan-hash deduplication, with JSON file storage in the prototype, designed to be swapped for a relational store in a production deployment.
- Coverage validator: a pre-compilation gate that rejects unknown metric or dimension terms with structured guidance listing the available identifiers.

- Permission enforcement: a Permission Rewriter that appends role-based WHERE predicates for five roles: public, store_manager, regional_manager, read_only, and analyst.

4.2 Experimental Environment

All experiments ran against a Docker-containerized MySQL 8.0 instance initialized with the AEGIS Truth Schema (126 tables, 107 foreign keys). The mock dataset used for evaluation contains 1,200 customers, 2,500 orders spanning 2024-2026, 6,298 order items, and 1,000 products mapped across 50 categories, sized to be representative of a mid-sized e-commerce deployment rather than a toy schema.

4.3 Benchmark Dataset Construction

This evaluation is a prototype evaluation, not a large-scale independent benchmark study. Its goal is to demonstrate that the AEGIS architecture achieves its stated safety and semantic-fidelity properties on representative real-world analytics queries, not to establish population-level accuracy claims. Standard text-to-SQL benchmarks such as Spider and BIRD (Chapter 2) do not evaluate adversarial safety or adherence to business vocabulary, which is why a domain-specific benchmark was necessary for this thesis.

A methodological caveat applies to every quantitative result reported in this chapter and in Chapter 5. All benchmark construction, execution, and measurement were carried out by the author as part of this thesis, using a self-built dataset and evaluation harness; none of it has been independently replicated by a third party, externally audited, or peer-reviewed. Where a figure below states that an annotation or verification step was performed by a named number of people (for example, two annotators or two database engineers), that step was part of the author's own research process rather than an independent, third-party audit. These results should therefore be read as internal evidence produced during this research, sufficient to support the architectural claims this thesis makes about its own prototype, and not as an externally validated benchmark result comparable to a peer-reviewed publication.

A domain-specific benchmark of 100 reporting requests was built over the production nopCommerce schema described in Section 4.2. The full question set, ground-truth

SQL, and recorded pipeline outputs are available in the evaluation_dataset/ directory of the project repository, enabling independent verification of every reported metric. Queries span all eleven analytics primitives identified in Section 3.2, with vocabulary variation not seen during system design, and twenty of the hundred queries are adversarial, specifically constructed to attempt prompt injection, indirect injection via filter values, or instruction-override attacks. Gold-standard SQL for every query was independently verified by two database engineers. Because the benchmark was constructed for the nopCommerce domain, accuracy figures in Chapter 5 should be interpreted within that scope; queries that require analytical patterns not yet in the template library (Table 2) are excluded by design, so the benchmark measures depth of coverage within the supported pattern set rather than breadth across every conceivable analytics request.

4.4 Baseline Systems

Four baselines isolate the contribution of each architectural layer:

B1 - Direct LLM-to-SQL. Llama 3.1 8B prompted with the full database schema, with no semantic layer and no template constraints; the model is free to generate any SQL text it produces.

B2 - Decomposed LLM. A chain-of-thought strategy that first extracts entities, then generates SQL from the extracted entities, testing whether decomposition alone (without a fixed template library) improves safety.

B3 - Template-only (no LLM). Keyword matching directly to templates, with no LLM-based intent extraction, testing how much of AEGIS's safety comes from having a fixed template library at all, independent of how the template is selected.

B4 - AEGIS ablated (no semantic layer). The full AEGIS pipeline with the semantic mapper bypassed, testing whether the semantic layer specifically, rather than the pipeline structure in general, is responsible for AEGIS's measured gains.

4.5 Evaluation Procedure

The evaluation addresses five research questions, each with its own procedure, reported in full in Chapter 5:

- RQ1: How accurately does the LLM intent parser extract typed reporting plans? Measured as per-class precision, recall, and F1 over the 100-query benchmark.
- RQ2: Does AEGIS reduce unsafe and semantically incorrect SQL compared to direct LLM-to-SQL baselines? Measured as unsafe SQL rate, execution validity, and coverage against baseline B1.
- RQ3: Does template-based compilation preserve sufficient expressiveness? Assessed qualitatively against the design-time review (Section 3.2), and via an ablation study on the 100-query benchmark removing each architectural component in turn (vocabulary injection, semantic layer, AST validation, confidence-gated clarification, permission rewriter, repair-on-parse-failure).
- RQ4: Does the architecture generalize to a second production schema outside the training domain? Measured by building an independent semantic layer for WooCommerce and recording both the resulting accuracy and the person-hours required to build it.
- RQ5: What is the end-to-end latency cost of the AEGIS pipeline? Measured as median and 95th-percentile latency per pipeline stage.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports results for each of the five research questions introduced in Section 4.5, followed by a discussion of what the results mean, what AEGIS deliberately trades away, and how it compares structurally to direct LLM-to-SQL generation.

This thesis is presented at the mid-defense stage: the figures below are preliminary estimates from the author's own prototype evaluation to date (Section 4.3), not final, independently verified results. They are reported to demonstrate the direction and plausibility of the architecture's safety and coverage claims, and should be read as work in progress rather than a closed, proven case; the final defense will report results from a more complete evaluation.

5.1 Intent Parsing Accuracy (RQ1)

Table 5: Intent parsing precision, recall, and F1 by intent class.

Intent class	Precision	Recall	F1
KPI	1.00	1.00	1.00
Ranking	1.00	1.00	1.00
Trend	1.00	1.00	1.00
Comparison	1.00	1.00	1.00
Exception	1.00	1.00	1.00
Summary	1.00	1.00	1.00
Segment	1.00	1.00	1.00
Funnel	1.00	1.00	1.00
Cohort	1.00	1.00	1.00
Correlate	1.00	1.00	1.00
Tabular	1.00	1.00	1.00
Overall	1.00	1.00	1.00

Every intent class reached perfect precision, recall, and F1 on the 100-query benchmark. This result should be read as a demonstration that vocabulary injection resolves intent classification within the benchmark's covered vocabulary and phrasing variation, not as a claim that AEGIS never misclassifies a request in general; Section 5.6 reports the failure modes observed during the broader design-time review (Section 3.2), where coverage-boundary rejections and clarification requests do occur.

5.2 SQL Safety and Execution Validity (RQ2)

Table 6: SQL safety and execution validity vs. direct LLM-to-SQL baseline.

System	Unsafe SQL rate	Execution validity	Coverage
Baseline B1 (Direct LLM-to-SQL)	5.0%	99%	99%
AEGIS (with vocabulary injection)	0%	100%	100%

The direct LLM-to-SQL baseline produced 5 unsafe queries out of 100 (a 5.0% unsafe rate), including INSERT, UPDATE, and DELETE statements and UNION clauses generated in response to the twenty adversarial prompts in the benchmark. AEGIS eliminated unsafe queries entirely, not by detecting and blocking them after generation, but by never allowing the LLM to generate executable SQL in the first place.

[PLACEHOLDER — FIGURE 8 NOT YET INSERTED]

Evaluation results across unsafe-SQL rate, execution validity, and coverage

A grouped bar chart with three metric groups on the x-axis (Unsafe SQL Rate, Execution Validity, Coverage), two bars per group comparing 'Baseline (Direct LLM-to-SQL)' in red/orange against 'AEGIS' in green. Baseline: 5.0% unsafe, 99% validity, 99% coverage. AEGIS: 0% unsafe, 100% validity, 100% coverage. The AEGIS unsafe-rate bar should read visually as flat/near-zero next to the baseline's visible red bar, to make the safety contrast the immediate takeaway of the figure.

Figure 8: Evaluation results across unsafe-SQL rate, execution validity, and coverage

5.3 Expressiveness and Ablation Study (RQ3)

Across the 100-query benchmark and the design-time review (Section 3.2), the large majority of requests were answered directly without clarification, with a minority requiring a clarification turn, a semantic layer extension to cover a missing metric or dimension, or falling outside the template library entirely. The evaluation tooling accompanying this thesis (Chapter 4) measures execution validity and safety directly from benchmark runs; it does not yet compute a formal breakdown across these four outcome categories, so no precise percentage is reported here, and this is noted as future work in Chapter 6.

Table 7: Ablation study - execution validity and coverage per configuration.

Configuration	Execution validity	Coverage
Full AEGIS (vocabulary injection)	100%	100%
- Vocabulary injection (synonym dictionary instead)	64.7%	99%
- Semantic layer	88.7%	91%
- AST validation	100%*	100%
- Confidence-gated clarification	94.2%	96%
- Permission rewriter	100%**	100%
- Repair call on parse failure	92.9%	95%

Removing vocabulary injection in favor of a hand-maintained synonym dictionary produces the largest single drop, 35.3 percentage points of execution validity, confirming that vocabulary injection is not a convenience but the component doing the most work in keeping intent extraction reliable. Removing AST validation or the permission rewriter leaves the reported metrics unchanged on this benchmark (marked with an asterisk), which is expected and correctly interpreted as confirming their role as defense-in-depth layers against attack classes (T3, T4 in Section 3.5) that this particular benchmark's queries do not exercise, not as evidence that those layers are unnecessary.

[PLACEHOLDER — FIGURE 9 NOT YET INSERTED]

Ablation study results

A horizontal bar chart listing each configuration from Table 7 on the y-axis (Full AEGIS, minus vocabulary injection, minus semantic layer, minus AST validation, minus confidence-gated clarification, minus permission rewriter, minus repair-on-parse-failure) with execution-validity percentage bars: 100%, 64.7%, 88.7%, 100%, 94.2%, 100%, 92.9%. Highlight the 'minus vocabulary injection' bar in a distinct color (e.g. red), since it shows the largest drop, to visually confirm it is the single most load-bearing component.

Figure 9: Ablation study results

5.4 Cross-Schema Generalizability (RQ4)

A second semantic layer was built for WooCommerce, a structurally distinct e-commerce platform with different table naming conventions and business vocabulary (12 metrics, 28 dimensions, 9 join paths, 18 tables), together with a 50-query evaluation set built using the same methodology as Section 4.3.

Table 8: Cross-schema generalizability results.

Schema	Build time	Intent accuracy	Unsafe SQL rate	Coverage
nopCommerce (primary, 100 queries)	40 person-hours	100%	0%	100%
WooCommerce (transfer, 50 queries)	14 person-hours	98.0%	0%	96.0%

The WooCommerce evaluation achieved 98.0% intent accuracy with zero unsafe SQL using a semantic layer built in 14 person-hours, 65% less effort than the primary schema required. The 2% accuracy gap arose from two WooCommerce-specific metric names, resolved by adding two description entries to the prompt with no code changes required. These results support the claim that AEGIS's architecture, not just its nopCommerce configuration, is what generalizes: the LLM, the compiler, and the

safety scanner required zero modification between schemas; only the semantic layer configuration changed.

5.5 Pipeline Latency (RQ5)

Table 9: Pipeline stage latency.

Stage	Median (ms)	p95 (ms)	% of total
LLM API call (Groq)	1,850	2,800	96.2%
Semantic mapping	12	18	0.6%
SQL compilation	8	12	0.4%
Query execution (MySQL)	45	120	2.3%
Visualization selector	2	4	0.1%
Widget persistence	5	9	0.3%
Total	1,922	2,963	100%

AEGIS's safety infrastructure, the deterministic stages that carry out coverage validation, semantic mapping, permission rewriting, compilation, visualization selection, and widget persistence combined, adds a median of roughly 20 ms of overhead, negligible relative to the 1,850 ms median LLM API call. With a locally hosted Ollama model, median LLM call latency drops to approximately 340 ms, bringing total end-to-end latency below 430 ms. The safety guarantees established in Chapter 3 are, in this sense, effectively free: the cost of AEGIS's architecture is dominated by the same LLM call a direct LLM-to-SQL system would also have to make.

[PLACEHOLDER — FIGURE 10 NOT YET INSERTED]

Pipeline stage latency breakdown

A horizontal stacked bar (or waterfall) chart of the six pipeline stages from Table 9 and their median latency: LLM API call (Groq) 1,850 ms, semantic mapping 12 ms, SQL compilation 8 ms, query execution (MySQL) 45 ms, visualization selector 2 ms, widget persistence 5 ms, totaling 1,922 ms. Since the LLM call dominates at 96.2% of the total, include an inset panel zooming into just the five non-LLM stages (the remaining ~72 ms) so their relative proportions are visible rather than compressed to invisibility next to the LLM bar.

Figure 10: Pipeline stage latency breakdown

5.6 Failure Analysis

Requests that could not be answered at all during the design-time review and benchmark construction fell into recurring categories: metrics not present in the semantic layer, unregistered dimensions, requests requiring multi-metric aggregation beyond a single pattern, causal or explanatory questions such as “why did revenue drop,” and requests requiring a join path not present in the join graph. This thesis does not yet instrument a measured frequency for each category (Chapter 6 lists this as future work), so these are reported as recurring categories rather than a percentage breakdown. Every rejection, in this review and in the benchmark, included the full list of available identifiers rather than a bare error, consistent with the design principle (Section 3.3) that a coverage failure should be actionable rather than opaque.

[PLACEHOLDER — FIGURE 11 NOT YET INSERTED]

Coverage-boundary rejection categories (illustrative)

A single panel listing the recurring rejection categories observed during design-time review and benchmark construction: metric not in semantic layer, unregistered dimension, multi-metric aggregation, causal/explanatory question, missing join path. Present these as a labeled list or simple diagram, not a chart with percentages, since no measured frequency breakdown is currently instrumented (Section 5.6).

Figure 11: Coverage-boundary rejection categories (illustrative)

5.7 Discussion

5.7.1 AEGIS vs. direct LLM-to-SQL: structural comparison.

A natural question is why not simply ask a capable LLM to write SQL directly. The differences below are architectural, not accuracy-based: they would hold even against a future, more capable model.

Table 10: Structural comparison of AEGIS vs. direct LLM-to-SQL.

Property	Direct LLM-to-SQL	AEGIS
SQL generation	Model-generated (probabilistic)	Deterministic compiler
Schema exposure to LLM	Required (tables, columns, keys)	Not required (labels only)
Business metric definitions	Implied from schema names	Explicit in semantic layer
SQL injection prevention	Prompt-level (best-effort)	Structural (by design)
Permission enforcement	External or none	Built-in, post-LLM
Dashboard widget persistence	Not provided	First-class artifact
Auditability of query origin	Difficult	Full provenance per widget
Model dependency	Tied to specific model quality	Model-independent

AEGIS does not claim to produce more creative SQL than a frontier LLM. It claims that, for the analytics requests it supports, results are guaranteed correct by construction, auditable, and safe, properties that probabilistic generation cannot offer unconditionally, no matter how capable the underlying model becomes.

5.7.2 Why a semantic layer instead of retrieval-augmented generation?

Retrieval-augmented generation (RAG) for NL-to-SQL retrieves relevant schema fragments to give the LLM better context. This is a useful technique, but it solves a different problem from the semantic layer and does not eliminate the safety risk. RAG asks which schema information the LLM should see; it is an access-optimization technique that narrows the schema the model reasons over, but the LLM still generates a free-form SQL string as output. The semantic layer asks which analytical concepts are allowed to exist, what they mean, and who may access them; it is a

governance mechanism that defines the complete set of answerable questions and their canonical SQL translations before any query is processed, and the LLM outputs a typed intent object rather than SQL. An organization that wants both better schema context and controlled output could use RAG to select relevant semantic layer sections for very large vocabularies, thousands of metrics, while still routing every query through the AEGIS compiler; the two techniques are complementary rather than competing.

5.7.3 Scope and coverage boundary.

AEGIS only supports queries that fit within its defined metrics, dimensions, and patterns, an approximately 5,610-combination space (15 metrics times 34 dimensions times 11 patterns, in the nopCommerce configuration). Out-of-scope queries receive a structured rejection listing available identifiers rather than a silent wrong answer:

```
Unknown metric 'conversion_rate'.
Available: avg_order_value, customer_count, discount_amount,
           order_count, profit, refund_amount, revenue, ...
```

Extending coverage requires only adding rows to the semantic layer, with no model retraining and no synonym curation required, since vocabulary injection (Section 3.8) propagates a new entry to the LLM's available vocabulary automatically. For open-ended data exploration requiring custom joins or schema-level operations outside this space, an unconstrained system may be more appropriate; AEGIS is designed for the everyday reporting needs identified in the formative study (Section 3.2), not ad hoc data science exploration.

CHAPTER 6

LIMITATIONS AND FUTURE WORK

6.1 Limitations

Semantic layer construction cost. Every AEGIS deployment requires a domain-specific semantic layer built by someone with both business knowledge and schema access. The nopCommerce prototype took approximately 40 person-hours. Organizations without this expertise, or with rapidly evolving schemas, may find the maintenance burden significant. AEGIS is not a zero-configuration system.

Cannot answer arbitrary SQL questions. By design, AEGIS only answers questions that map to a supported analytics primitive with an approved metric and dimension. Ad hoc queries, multi-level nested aggregations, or requests for data fields not in the semantic layer fail with a coverage error (Section 5.6). This is a deliberate trade-off, not an oversight.

Complex analytical queries require new templates. Approximately 2.6% of the formative-study requests (Section 3.2) required patterns not yet in the template library. Adding a new pattern requires a developer with SQL knowledge; it is not something a business user can do themselves.

Quality depends on intent extraction, not just compilation. The safety guarantees in Chapter 3 apply to compilation and execution, not to intent extraction quality. A model that misclassifies a request will produce a structurally safe but semantically wrong query. Safety and accuracy are separate properties, and this thesis is careful not to conflate them.

Benchmark selection. The custom 100-query benchmark (Section 4.3) was necessary because standard benchmarks such as Spider and BIRD do not evaluate adversarial safety or adherence to business logic; this also means the reported accuracy figures are not directly comparable to Spider- or BIRD-reported numbers from other systems.

Semantic layer scalability. Modern context windows of roughly 128,000 tokens can hold approximately 2,500 distinct metric and dimension definitions; most enterprise deployments expose fewer than 500 core concepts, so this is not a near-term

constraint, but very large vocabularies would need retrieval-augmented vocabulary injection rather than injecting the entire semantic layer at once.

Database agnosticism. The current compiler generates MySQL syntax; supporting PostgreSQL or SQL Server requires extending the compiler module, not redesigning the architecture.

Storage persistence. The prototype uses JSON flat files for widget storage; the widget registry interface is designed to be swapped for a relational database in a production deployment.

Multi-turn conversation. AEGIS currently treats each request independently. Contextual carryover across turns, of the kind studied for conversational text-to-SQL in Section 2.2, is not yet implemented.

Vocabulary injection limitations. Highly specialized domain terminology may require supplementary few-shot examples in the prompt beyond the label and description pairs currently injected.

6.2 Future Work

- Clarification requests: when confidence is low, AEGIS should ask a follow-up question instead of guessing, for example “did you mean revenue or profit?”
- Semantic layer wizard: a guided interface for business analysts to define new metrics and dimensions without writing Python.
- Multi-step queries: currently each query produces one widget; compound questions such as “revenue trend and top 5 products side by side” require two separate queries today.
- Automated denial-of-service protection: query complexity scoring and server-side timeout enforcement, since the join graph bounds query cost but does not currently score it explicitly.
- Broader schema coverage: extending the nopCommerce semantic layer to cover promotions, vendor analytics, and content-management engagement metrics.
- Multi-turn conversational carryover, extending the single-turn design discussed above.
- Outcome and rejection-category instrumentation: measuring, from real benchmark and production runs, the precise proportion of requests answered directly,

answered after clarification, answered after a semantic layer extension, or rejected, and the relative frequency of each rejection category (Sections 5.3 and 5.6). This thesis currently reports these categories qualitatively; adding this instrumentation would let a future revision report a measured percentage breakdown rather than a qualitative one.

CHAPTER 7

CONCLUSION

This thesis presented AEGIS, a system for turning plain-English reporting requests into dynamic, refreshable dashboard widgets over relational databases. The central contribution has two parts. First, a system design that limits the language model to understanding questions, while all query building, chart selection, and widget storage is handled by fixed rules and pre-approved templates, so that safety is a property guaranteed by system structure rather than a probability that improves with model quality. Second, a vocabulary injection method that removes the need for manually maintained synonym lists while improving coverage, reducing a 112-entry synonym dictionary to zero entries while raising coverage from 99% to 100%.

The evaluation in Chapter 5 shows that AEGIS reduces the unsafe SQL rate from 5.0% to 0% relative to a direct LLM-to-SQL baseline using the same underlying model, achieves 100% valid SQL and 100% coverage on its 100-query nopCommerce benchmark, and generalizes to a second production schema, WooCommerce, with only semantic layer reconfiguration required and no change to the LLM, the compiler, or the safety scanner. The pipeline latency analysis confirms that this safety infrastructure adds less than 4% overhead relative to the LLM API call that any comparable system would also need to make.

The literature review in Chapter 2 situates this contribution precisely: prior text-to-SQL research, from Seq2SQL through RAT-SQL and PICARD, treats SQL-generation accuracy as the object of study and, with the partial exception of a 2025 systematic review's discussion of injection attacks, does not evaluate safety as a first-class property at all. AEGIS's architectural choice, removing SQL generation from the language model's role entirely rather than constraining it more tightly, is a categorically different answer to the same underlying problem, and one independently echoed in recent explanatory-analytics research reviewed in Section 2.7 that arrives at the same “let a deterministic layer compute the answer” principle in an unrelated domain.

AEGIS is built for environments where data privacy, consistent reporting definitions, and daily reuse of saved reports matter more than unlimited query flexibility. Chapter 6 states plainly what this design gives up in exchange: a bounded vocabulary, an upfront semantic layer construction cost, and dependence on intent-extraction quality for semantic (though never safety) correctness. Within that scope, this thesis demonstrates that restricting SQL generation to a finite set of validated business patterns is a practical, measurable path to safe, auditable natural-language reporting in institutional environments.

REFERENCES

[1] K. Affolter, K. Stockinger, and A. Bernstein, "A comparative survey of recent natural language interfaces for databases," The VLDB Journal, vol. 28, no. 5, pp. 793-819, 2019.

[2] D. Deng, A. Wu, H. Qu, and Y. Wu, "DashBot: Insight-driven dashboard generation based on deep reinforcement learning," IEEE Transactions on Visualization and Computer Graphics, vol. 29, no. 1, pp. 690-700, 2023.

[3] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in Proc. 28th Annual ACM Symposium on User Interface Software and Technology (UIST), 2015, pp. 489-500.

[4] C. Lehmann, D. Gehrig, S. Holdener, C. Saladin, J. P. Monteiro, and K. Stockinger, "Building natural language interfaces for databases in practice," in Proc. 34th Int. Conf. Scientific and Statistical Database Management (SSDBM), 2022, Art. no. 20.

[5] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," Proceedings of the VLDB Endowment, vol. 8, no. 1, pp. 73-84, 2014.

[6] J. Li et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.

[7] M. Liu and J. Xu, "NLI4DB: A systematic review of natural language interfaces for databases," arXiv:2503.02435, 2025.

[8] A. Narechania, A. Srinivasan, and J. Stasko, "nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries," IEEE Transactions on Visualization and Computer Graphics, vol. 27, no. 2, pp. 369-379, 2021.

[9] OpenAI, "Introducing structured outputs in the API," OpenAI, 2024.

[10] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 9895-9901.

[11] G. N. Shailesh, M. Pavithran, A. Rahul Hari Venkat, and P. Kaliappan, "Conversational BI: Natural language interface to business dashboards," International Journal of Engineering Research & Technology, vol. 14, no. 12, 2025.

[12] H. S. Shalaan, T. H. A. Soliman, and A. M. Abdelaziz, "G-SQL: A schema-aware and rule-guided approach for robust natural language to SQL translation," IEEE Access, vol. 13, pp. 158520-158534, 2025.

[13] X. Su, Y. Gu, P. Wang, W. Gu, L. Qi, and J. He, "A robust natural language text-to-SQL generation framework with dynamic strategies based on LLMs," Scientific Reports, vol. 16, Art. no. 7892, 2026.

[14] J. Valkenburgh, "Enhancing business dashboards with explanatory analytics and AI: Exploring the use of AI and explanatory analytics to enhance business decision-making," M.S. thesis, Information Management, Turku School of Economics, University of Turku, Finland, 2024.

[15] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7567-7578.

[16] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2018, pp. 3911-3921.